AutoHealOps

A plain-language guide

What the project does
How it works
Why it is needed

Manikant Kumar
Project guide and abstract
25 October 2026

autohealops.vercel.app

Public website: interactive browser simulation
Local repository: API, storage, optional ML and cluster integration

01 Abstract and purpose

AutoHealOps is a software operations project that helps people understand the health of an application and review a controlled response when something looks wrong. It brings measurements, fault explanations, incident records and recovery decisions into one dashboard.

The project observes signals such as processor use, memory consumption, restart counts and response delay. Depending on the selected setup, it checks those measurements with transparent rules, a trained anomaly detector, or both. It records the evidence and supports a review process before a permitted recovery action.

Its purpose is to make reliability work easier to explain, demonstrate and inspect, and to automate supported recovery runbooks for explicitly enabled workloads. Someone can follow the reason for an alert, the target of a proposed action, the approval decision and the outcome. This supports learning and engineering evaluation without claiming that every problem has an automatic cure.

The important distinction

The public website is a browser simulation. It has no connected backend, trained machine-learning service, database server or real Kubernetes cluster. Anyone can explore it without a username or password. Each visitor has a separate browser session.

The repository also contains a fuller local system. That setup can run the FastAPI backend, persistent storage and an optional trained detection service. A deliberate live configuration can connect a restricted Kubernetes namespace and supported Deployment actions.

Who this guide is for

Students, project evaluators, developers and readers who know little about cloud operations can start here. Technical readers can use the linked architecture, API, AI and security documents for implementation detail.

What the project does not promise

The project does not provide proven production availability, automatic root-cause diagnosis, a calibrated probability of failure or an exact time until a service fails. A successful simulation demonstrates the workflow, not a real infrastructure repair.

02 Why this project is needed

Imagine an online shop whose checkout becomes slow. Customers see the delay, but the technical team must work out what is happening behind the screen. The processor may be busy, memory may be rising, containers may keep restarting, or requests may take longer than usual. These are clues. A clue alone does not prove the cause.

The practical problem

Health measurements and operational decisions can live in different places. One view may show resource use, another may show alerts, and an operator may make a recovery decision separately. That makes it harder to explain why an action was appropriate or to reconstruct the sequence later.

AutoHealOps connects those parts in a visible workflow. It displays observations, explains why a check triggered, records an incident and tracks a controlled action. The intended benefit is clearer investigation and a more reviewable response. The project has not measured a percentage reduction in downtime or operating cost.

What Kubernetes already does

Kubernetes already manages running application copies and can restart certain failed containers or replace missing pods according to its configuration. AutoHealOps adds a project-level layer for evidence, explanation, policy, approval and record keeping around supported operations. It works with Kubernetes capabilities rather than replacing the platform.

Why a simulation helps

A student or reviewer can demonstrate a CPU spike and review a recovery recommendation without renting cloud machines or risking a real workload. The same idea can then be studied in the local backend. Real-cluster use requires additional services, permissions and testing.

Need	How the project addresses it
Understand a symptom	Charts, workload inventory and a readable detection explanation.
Review a proposed response	Explicit target, rationale, approval and policy checks.
Learn what happened	Before/after observations, outcome and an audit trail.

03 The words, without the jargon

Term	Everyday meaning
Container	An application packaged with the software it needs to run.
Pod	A small running unit in Kubernetes that holds one or more containers.
Deployment	Instructions that keep a chosen number of application copies running.
Cluster	The group of machines that runs the applications.
Namespace	A named area used to group and limit access to related resources.
Telemetry or metrics	Measurements such as resource use, restart counts and response delay.
Anomaly	An observation that looks unusual compared with a baseline or a rule.
Remediation	An action intended to recover from a problem.
Audit trail	A record of who decided or acted, why, and what happened.

A building-manager analogy

Think of a cluster as a building that hosts several businesses. A namespace is a named area. A Deployment describes how many workers should be present. Pods are the running work units. Measurements are the readings on a manager's control board.

AutoHealOps resembles a control desk that collects those readings, identifies unusual conditions and records a carefully limited response. The analogy is a memory aid. Real Kubernetes units follow software rules and do not map exactly to rooms or people.

04 What each component contributes

Part	Its role in the local system
React dashboard	The interface for viewing measurements, resources, explanations, incidents and recovery decisions. Vite builds the application.
FastAPI backend	Coordinates API requests, collection, incident records, policy checks and the execution workflow.
AI service	Loads a compatible trained detector when available and exposes validated inference and model status.
SQLite or MongoDB	Stores telemetry, alerts, anomaly evidence, actions and audit events. Native demo uses SQLite by default. Compose supplies MongoDB.
Prometheus and Grafana	Optional measurement querying and platform monitoring. Workload metrics need suitable exporters and instrumentation.
Kubernetes adapter	Restricted inventory, manual actions and explicitly enabled autonomous runbooks for exact Deployment targets.

How these pieces relate

The dashboard requests data from the backend. A single collection loop samples the selected data source and records useful history. The backend asks the AI service to inspect complete inputs when it can, or uses labelled rules. Incidents and actions share correlation identifiers so readers can connect the evidence and decisions.

The public Vercel build runs a separate, explicit browser simulation. It does not call the local backend or open a WebSocket. This keeps a visitor's demonstration separate from real infrastructure.

05 How the response works

1. Observe

Collect available processor use, memory utilization, restart counts and network latency. In a demo, the generator supplies clearly labelled synthetic values. In a configured live setup, Prometheus and Kubernetes provide actual measurements and inventory.

2. Detect and explain

Check whether values or combinations of values look unusual. Show the supporting measurements, threshold comparisons and model identity. Missing live measurements remain unavailable. The system never substitutes random demo readings for a failed live integration.

3. Open an incident

Record the anomaly and an alert with severity, time, target context and a shared correlation identifier. Repeated observations of the same unresolved condition can share a deduplicated incident rather than creating an alert on every sample.

4. Review a recommendation

A recommendation identifies a supported action and an eligible target. Namespace-wide measurements cannot choose a repair target. Manual actions name a managed Deployment; autonomous runbooks use exact owned-Pod evidence for an opted-in target. An alert alone does not authorize a mutation.

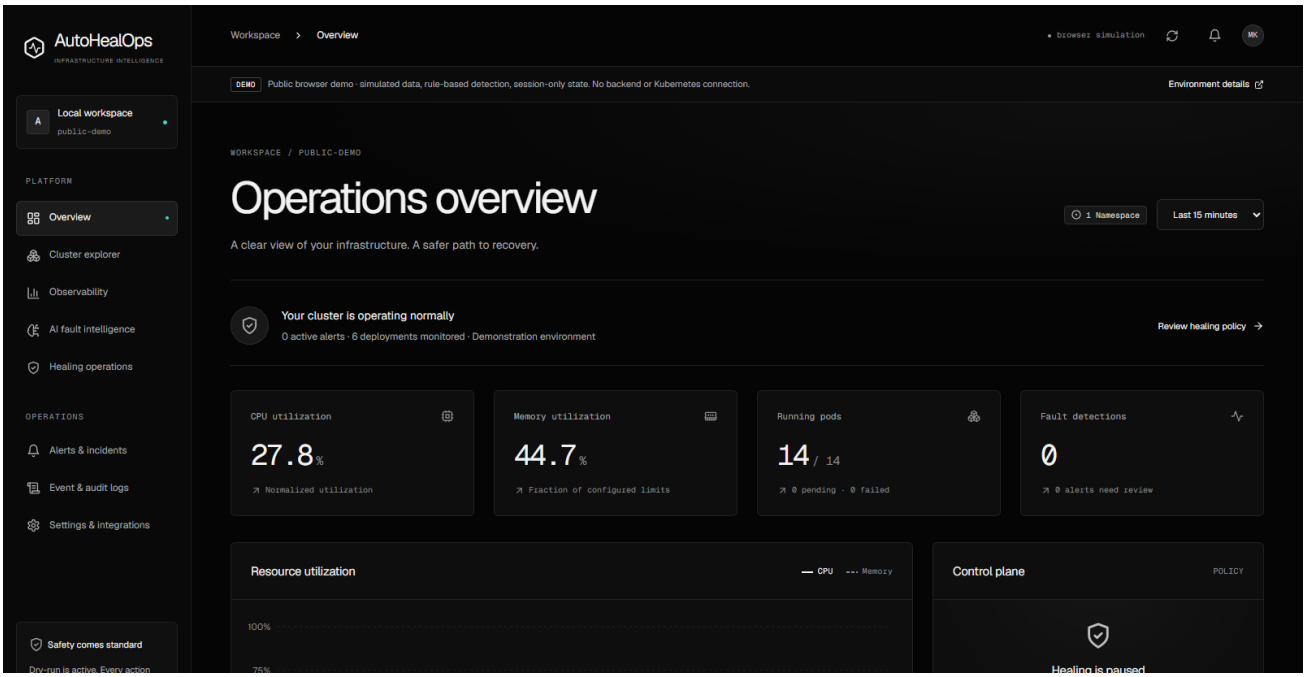
5. Apply policy and approval

The backend checks pause state, operating mode, ownership, replica bounds, previous attempts, cooldown and the action budget. Operator-created recommendations require approval. Opted-in live workloads can use confirmed autonomous runbooks without a per-action approval. Local startup begins paused and in dry-run mode.

6. Record and verify the outcome

The demo returns an explicitly simulated result. A live mutation must pass rollout-availability checks. The platform records failure if it cannot verify that result. Later telemetry helps determine whether the application's measurements actually improved. API acceptance and application recovery are separate observations.

06 What readers see in the dashboard



Upper portion of the actual AutoHealOps interface. All values shown here belong to the labelled demonstration.

View	Question it helps answer
Overview	What needs attention, and which data source am I seeing?
Cluster explorer	Which available workloads exist, and what details can I inspect?
Observability	How have the retained measurements changed?
AI fault intelligence	Why did a check trigger, and which model or rule produced it?
Healing operations	What action is recommended, who approved it, and what happened?
Alerts and incidents	Which issues are open, acknowledged or resolved?
Event and audit logs	Who acted, what was the reason, and which records belong together?
Settings and integrations	Which policy and services are active without exposing secrets?

07 The AI, explained honestly

Transparent rules

A rule can say that CPU above a documented threshold deserves attention. The explanation names the measured value and the threshold. The public browser demo uses this approach and clearly labels its results as rule based.

A learned anomaly detector

The local AI service can load a saved StandardScaler and Isolation Forest. The scaler prepares measurements in the same way during training and inference. Isolation Forest finds observations that differ from its training baseline. This project trains reproducibly on synthetic telemetry and saves model metadata.

Why the hybrid result matters

The final detector can flag either an unusual learned pattern or a threshold breach. If a threshold triggers while the learned model sees the full input as ordinary, the result still shows the original model decision and score. It does not pretend that the model found the fault. A joint-pattern anomaly can also occur without one individual threshold crossing.

Forecasting answers a narrower question

The local predictor compares a simple trend with a last-value baseline for CPU and memory one sample ahead. It evaluates holdout predictions in chronological order using only earlier observations. It reports error in resource-fraction units when a suitable sequence is available.

An anomaly score is not a failure probability. The project does not supply calibrated confidence, a failure probability or time-to-failure. Synthetic evaluation does not prove useful accuracy on a real cluster. The public website runs neither the trained model nor the forecasting service.

Input	Meaning
CPU and memory	Fractions from 0 to 1, shown as percentages in the interface.
Restarts	A count. Live collection uses the rounded increase over five minutes.
Network latency	Delay in milliseconds where the relevant measurement exists.

08 Why recovery needs guardrails

A recovery action can make a problem worse if it targets the wrong workload, repeats too often or assumes success without checking. AutoHealOps therefore limits the available actions and records the decision process.

Safeguard	Backend behavior
Safe initial state	Paused and dry-run by default. An operator must deliberately resume execution.
Approval	Manual requests require approval; configured autonomous runbooks use confirmed workload evidence instead.
Target boundary	Configured namespace and an explicitly managed Deployment label.
Supported scope	Deployment rollout restart or scaling within configured bounds.
Replica limit	Default maximum of 5 replicas. Scaling below 1 is not allowed.
Execution budget	One operation at a time, one attempt per action, and ten operations per hour.
Cooldown and replay	Default target cooldown of 300 seconds. Completed requests return the previous result rather than executing again.
Honest verification	Kubernetes API calls have timeouts. Rollout verification waits up to 45 seconds and can fail.
Restart behavior	Healing pauses after process restart. Interrupted executions become failed records for investigation.

The public showcase uses a shorter 30-second simulated cooldown and session-level limits. These demonstrate the idea and do not secure real infrastructure. The implementation has no automatic namespace, volume or database deletion, node drain or arbitrary-shell recovery operation.

Emergency pause blocks new actions. It cannot reverse a mutation that Kubernetes already accepted. Actual use still needs a reviewed test environment and appropriate permissions.

09 A manual CPU-spike example

This example uses the implemented synthetic scenario. It illustrates the sequence rather than describing an incident in a real shop.

The starting point

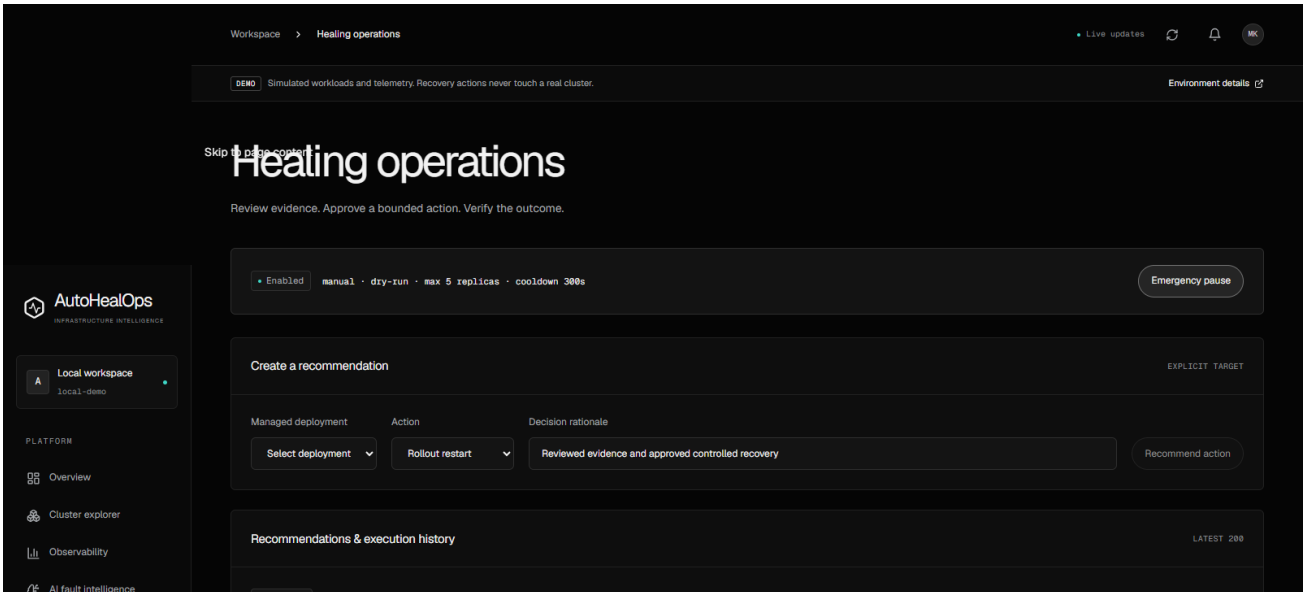
The dashboard shows changing healthy demo measurements. Recovery starts paused. A visitor or local demo operator chooses the CPU-spike scenario.

What changes

The scenario sets simulated CPU utilization to 96%. A threshold guard can explain that CPU exceeds its documented limit. The system opens an incident and proposes a recovery recommendation for the simulated checkout service.

What the operator reviews

The operator reads the evidence and rationale, checks the target and approves the recommendation. The operator then resumes the policy. Approval and resume serve different purposes: approving a particular action does not automatically unpause the whole recovery process.



Actual healing screen showing approval and simulated outcomes. The screenshot is evidence of the interface, not a real Kubernetes repair.

Choosing Simulate recovery restores healthy demo telemetry and records before/after observations. The exact later measurement depends on the demo generator. The result explicitly says simulated recovery. The audit trail remains, and the operator can acknowledge or resolve the incident separately.

10 Where the project can run

Mode	What is available	What it means
Public website	Browser-session history, synthetic telemetry, transparent rules and simulated recovery.	No login, backend database, trained model or Kubernetes connection. Each visitor has a separate simulation.
Local native demo	React, FastAPI, SQLite and an optional AI service with persisted model artifacts.	No real cluster required. Data and recovery remain explicitly simulated.
Local Compose demo	Containerized application, MongoDB and optional monitoring profile.	Requires Docker Desktop or a compatible Docker engine. It still defaults to demo and dry-run settings.
Configured live integration	Restricted inventory, exporter-backed metrics and manual or explicitly enabled autonomous Deployment actions.	Requires real services, strong credentials, scoped identity, deliberate configuration and validation.

Why the distinction matters

A website can explain a concept without operating a cluster. The public demo gives anyone a safe way to explore the interface, but it cannot prove that an application recovered in Kubernetes. A local backend adds persistence and service behavior. A live connection adds an infrastructure trust boundary.

The public site never falls back to a secret production connection. The normal backend build and the public showcase build remain separate. Browser state is bounded and belongs to the visitor's session. Reloading preserves that history and resets healing to paused.

Live integrations do not gain cluster-wide visibility automatically. The configured namespace limits what the service can inspect and mutate. Node inventory in the public demo is simulated.

11 A simple first demonstration

Open the live project

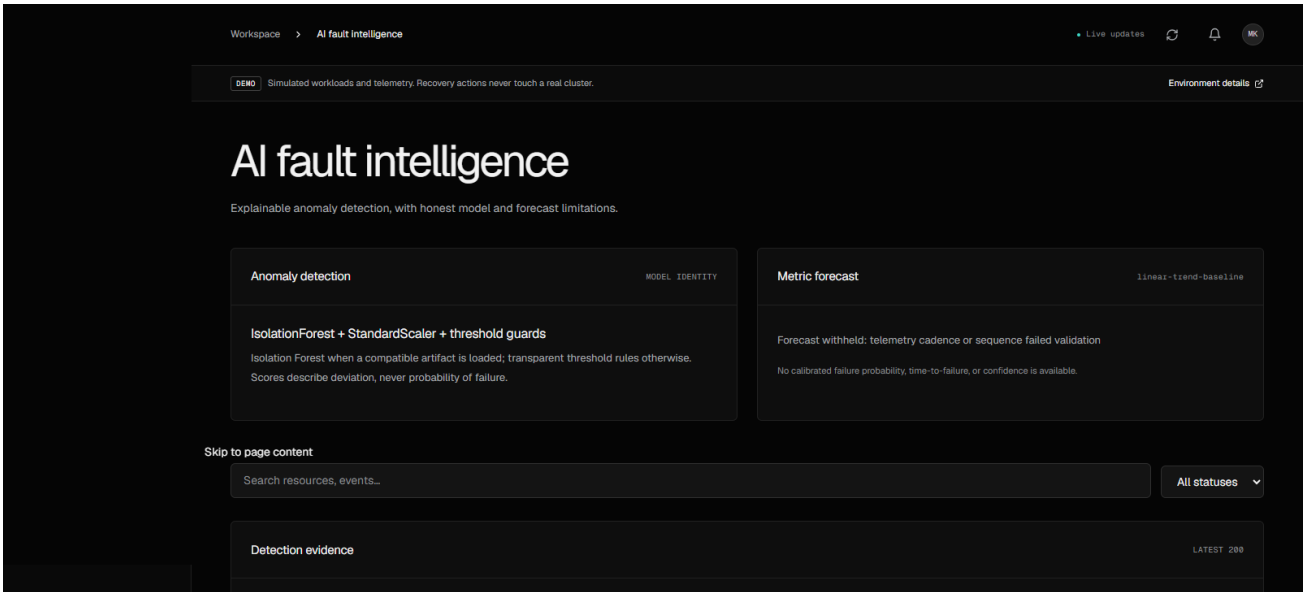
https://autohealops.vercel.app
Username: not required. Password: not required.

Follow the example

1. Open Overview and read the data-source banner.
2. Use Scenario lab to inject CPU, memory, restarts or latency.
3. Open AI fault intelligence and read the rule explanation.
4. Open Healing operations and review the proposed action.
5. Approve it, resume healing and choose Simulate recovery.
6. Compare before/after values, inspect audit records and resolve the incident when appropriate.

What a good demonstration explains

Explain the evidence before clicking recovery. Point out the target, the approval decision and the simulated outcome. Show that pausing blocks a new execution and that repeated actions respect a cooldown. Use Reset scenario to restore healthy telemetry while preserving the demonstration history.



Upper portion of the fault-intelligence view. Local screenshots can show hybrid inference. The public website labels its browser rules explicitly.

If a button is disabled, inspect the current policy or action state. A recommendation may need approval, healing may still be paused, or a cooldown may apply. Those checks are part of the workflow rather than evidence that a real recovery occurred.

12 Running the fuller local system

Native Windows route

The repository includes a native startup script for Python 3.12 and Node 22 or newer. It creates the local environment, installs the application dependencies, builds the frontend and launches the local services. The documented script forces a safe developer demo with SQLite and no real Kubernetes mutation.

From the repository root:
./scripts/start-native.ps1 -Install -BackendPort 18000 -AiPort 18001 -DashboardPort 13000

Dashboard: http://localhost:13000
API documentation: http://localhost:18000/docs
AI documentation: http://localhost:18001/docs

Docker Compose route

With Docker Desktop running in Linux-container mode, use ./scripts/setup.ps1. The core stack includes the frontend, backend, AI service and MongoDB. The optional monitoring profile adds Prometheus and Grafana. Setup preserves existing configuration and waits for health checks. Consult the setup guide for platform dependencies and troubleshooting.

Storage, identity and retention

The local backend stores metrics, anomalies, alerts, action records and audit events. SQLite uses bounded cleanup. MongoDB uses indexes and expiry rules. Default retention is seven days. Longer compliance retention needs a separate policy and archive.

The developer demo intentionally has a demo identity. Live or production settings require strong credentials and expiring JWTs with administrator and viewer roles. Secrets stay in private environment configuration or Secret references. The settings interface reports safe status rather than secret values.

Before connecting a real cluster

Supply MongoDB, Prometheus and suitable exporters. Choose a namespace and a least-privilege service account. Opt in only the target Deployment. Start with dry-run, examine the policy and test rejected or unavailable-service cases. Keep the backend at one worker and one replica until distributed coordination exists.

13 Benefits, limits and future work

What the current project can help with

It can make a reliability project easier to demonstrate and discuss. Readers can see how measurements become an explanation, an incident and a reviewed action. Developers can inspect the implementation and exercise safety checks. The repository provides a concrete base for further testing and learning.

What requires caution

Thresholds and synthetic training reflect an example baseline. Real workloads vary by application and operating conditions. A detector can miss a symptom or flag something harmless. A rule-triggered alert should not be presented as a learned-model success. A restart may hide a deeper issue or have no useful effect.

A verified rollout means the deployment reached the checked availability conditions. It does not by itself show that customer requests recovered or that the original cause disappeared. Later telemetry and application-level checks remain important.

Current engineering boundaries

The backend runs as one process. In-memory locks and scheduling do not coordinate multiple API instances. Notifications are optional and best effort. The project does not claim production high availability, an immutable compliance archive, a calibrated failure predictor or comprehensive automatic root-cause analysis.

Useful next steps

Collect real, labelled operational data and evaluate it in a separate chronological test period. Add durable action claims and shared event delivery before scaling the control plane. Define cluster-specific authorization before supporting several clusters. Test restore procedures, workload effects, external delivery and failure recovery in an appropriate environment.

These are future engineering tasks. A feature listed as a future task should never be read as already available. Tests and CI confirm specific behavior, not a guarantee of uptime or business results.

14 What autonomous recovery means

An administrator chooses the workloads and permitted policy in advance. Once automatic mode is resumed, the controller can confirm a supported fault, choose its runbook, execute a bounded action, check the result and record it without waiting for someone to approve each individual action.

An everyday analogy

Think of a building manager who has permission to call in one extra worker when a particular service is repeatedly overloaded. The manager checks the evidence first, stays within a staffing limit and checks whether the queue improves. If the check fails, the manager stops and asks for investigation. This is a memory aid, not a description of a real incident.

Live runbook	Condition and bounded response
CPU pressure	Maximum owned-container CPU above 80% of its limit: add one replica, up to the configured maximum.
Repeated restarts	More than five restarts across owned Pods in one minute: request a rollout restart. Restart evidence takes precedence.
Other symptoms	Memory and latency remain useful evidence. They do not independently authorize an automatic live repair.

Evidence identifies the workload

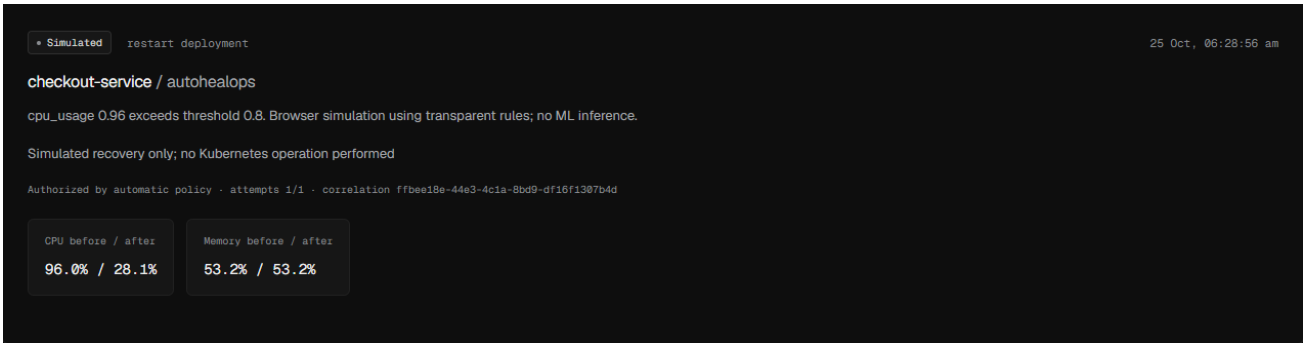
The adapter follows Deployment, ReplicaSet and Pod controller identities. It queries only exact owned Pods, checks their measurement coverage and resource limits, and rejects missing or stale evidence. Three distinct source samples must confirm the same runbook with stable workload identity. Reading the same scrape again does not count.

The live runbooks use transparent rules. The optional trained anomaly detector remains a separate capability; it does not identify root causes or choose arbitrary repairs. A CPU problem may not improve after scaling, and a bad image may keep failing after a restart. Verification therefore matters.

15 Enabling and demonstrating autonomy

Try the public simulation

Open the website without a login. In Settings select Autonomous recovery and save. In Healing operations choose Resume healing. Go to Overview and inject a fault. After three browser samples, the simulation recovers without an approval click and resolves the related incident. Inspect the action and audit history. Reloading pauses healing and restores manual mode.



Actual compiled browser interface after automatic simulated recovery. No backend or Kubernetes operation was involved.

Configure a live test target

The full backend requires MongoDB, Prometheus, suitable exporters, strong credentials and scoped Kubernetes identity. Use one backend worker and replica. Set ALLOW_AUTOMATIC=true, HEALING_MODE=automatic and exact AUTOMATIC_TARGETS, such as checkout-service. Both Deployment metadata labels, autohealops.io/managed and autohealops.io/autonomous, must be true.

The selected namespace must contain no HPAs. Paused or zero-replica targets are excluded. Do not opt in the control plane itself. Start with DRY_RUN=true and inspect the generated evidence. Real mutation requires a deliberate DRY_RUN=false configuration and restart. Startup always pauses healing; resume explicitly after review. The detailed repository runbook gives all settings and prerequisites.

16 Verifying recovery and stopping safely

The success check

The adapter first verifies rollout availability using observed generation and updated/available replicas. It then waits for two distinct healthy source observations after execution: CPU at or below 80%, memory at or below 85% and no more than five restarts in the one-minute window. Only then does an autonomous live action succeed and resolve its correlated incident.

The failure check

An unverified rollout, missing evidence or measurements that stay unhealthy records a failed action, leaves the incident open and pauses healing. A persisted per-target latch prevents another automatic attempt for the sustained fault. Two healthy source observations can re-arm the target; records follow configured retention. An operator can investigate and create a separate approved action when needed.

Boundaries remain enforced

Immediately before mutation the adapter rechecks namespace, both labels, UID, generation, paused state, replicas and HPA absence. A resource-version precondition rejects intervening Deployment changes. Manual and automatic actions share one execution lock, one attempt per action, a 300-second target cooldown and ten operations per hour. The default maximum is five replicas.

Rollout polling uses a 45-second window and five-second API request timeouts. Telemetry verification defaults to a 120-second polling window, plus bounded in-flight calls. A separate recovery task lets normal dashboard collection continue. Emergency pause prevents new operations but cannot undo an already accepted mutation. Restart begins paused and marks interrupted actions failed.

What has actually been checked

The backend suite passed 38 tests, including 19 autonomous controller and adapter contracts. The frontend suite passed 12 tests. Actual Edge checks covered automatic simulated recovery, no per-action approval, reload pause, keyboard focus and mobile width. The backend tests use controlled adapters and Prometheus responses.

This machine has no configured Kubernetes context and no running Docker engine. No real-cluster or production acceptance is claimed. Before real use, test exporter labels, ownership, RBAC, successful scale/restart, revoked permissions, HPA conflicts, missing data, failed rollout, persistently unhealthy telemetry and process restart in a disposable namespace.

17 Questions and further reading

Does the public demo need a password?

No. Open the Vercel URL directly. It is a separate browser simulation and exposes no infrastructure credentials.

Does every alert lead to an automatic repair?

No. A symptom can require investigation. The safe starting policy is paused and dry-run. Manual actions require a selected managed target and approval. Live autonomous actions require exact target configuration, fresh owned-Pod evidence and confirmed runbooks. Policy checks apply to both.

Can it predict the exact time a service will fail?

No. The local forecasting baseline estimates CPU and memory one sample ahead. Failure probability, calibrated confidence and time-to-failure are unavailable.

Can I connect it to production immediately?

The repository is a starting implementation. Use a deliberate test environment first and review identity, permissions, telemetry, storage, backups and workload behavior. The documentation does not claim that real-cluster production operation has been validated.

Where can I learn more?

Live interactive demo: autohealops.vercel.app

Source repository: manikantbindass / AutoHealOps

Repository reading order: README.md for entry points, docs/setup-guide.md for running locally, docs/architecture.md for components and trust boundaries, docs/ai-models.md for inference limits, docs/api-contract.md for endpoints, and docs/security-operations.md for operational controls, and docs/autonomous-recovery.md for the full autonomous procedure.

This guide describes source snapshot e9864e0 and the documentation available on 25 October 2026. The repository may change after this edition. The companion PowerPoint explains the same story with presenter notes.

Author: Manikant Kumar. Screenshots show the actual project with labelled demonstration data. No production incident, measured financial benefit or guaranteed reliability outcome is implied.